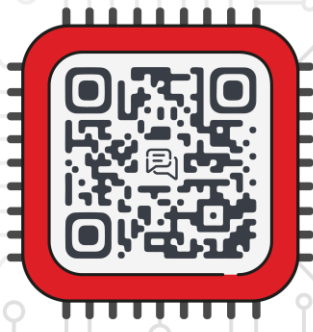




2



25



3750 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs;

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

Perceptron

عملي مشترك



الشبكات العصبونية

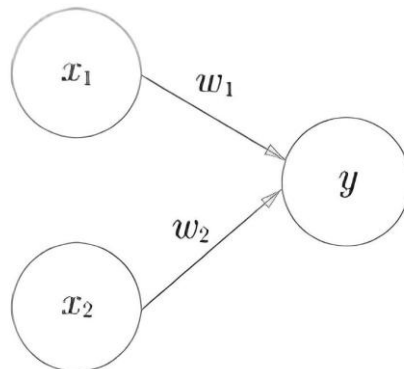
الشبكة العصبية (Neural Network)

تخيل أنك تريد تعليم طفل كيف يتعرف على القطعة، فبدلاً من أن تقدم له قائمة من القواعد المعقدة، ستقوم بعملية أبسط وأكثر فعالية: ستعرض عليه مئات الصور، وفي كل مرة تقول له: "هذه قطعة" أو "هذه ليست قطعة". مع الوقت، سيقوم دماغ الطفل تلقائياً باستخلاص الأنماط والميزات التي تميز الشيء (القطعة) دون أن تتمكن أنت أو هو من صياغة هذه القواعد بشكل صريح.

الشبكة العصبية هي النسخة الحاسوبية لهذه العملية؛ هي نظام حسابي مستوحى من بنية الدماغ البشري، مهمته ليس أن يتم برمجته بقواعد صارمة، بل لكي يتعلم هذه القواعد بنفسه من خلال عرض كميات هائلة من الأمثلة والبيانات. إنها تتكون من وحدات بسيطة (الموزونات) مترابطة، وقوتها الحقيقية تكمن في كيفية تعلم هذه الوحدات بالتعاون معاً لحل مشاكل معقدة.

البرسيبترون (Perceptron) :

- أبسط وحدة بناء في الشبكة العصبية.
- اخترعه فرانك روزنبلات في عام 1957 في مختبر كورنيل للطيران.
- هو نموذج رياضي يحاكي إلى حد كبير عمل الخلايا العصبية (النيورون)، حيث تستقبل الخلايا العصبية إشارات كهربائية وتقوم بمعالجتها ثم تقرر إذا كانت ستمرر إشارة أو لا.
- يتكون من مدخل واحد أو أكثر، ومعالج، ومخرج واحد فقط.



المدخلات (Inputs) :

- يستقبل إشارة واحدة أو أكثر، حيث كل إشارة هي قيمة ثنائية (Binary): إما 1 (تدفق إشارة) أو 0 (لا يوجد تدفق).
- المدخلات هي المعلومات التي يتم إدخالها إلى الخوارزمية: (Feature كمية المعلومات الموجودة ضمن العنصر الواحد).

الفرق بين الخصائص (Features) و Instance :

الخصائص (Features)	Instance
هو عمود في جدول البيانات، خاصية أو صفة نصف بها العنصر.	هو صف في جدول البيانات، مجمع لكل الخصائص مع بعضها.

مثال: لدينا جدول البيانات التالي الذي يعبر عن أسعار المنازل :

هل يوجد حديقة؟	المساحة	عدد الغرف	السعر	Features
0	150	2	100000	Instance
1	200	3	150000	Instance

لنقوم بتحليل الجدول :

- **خصائص (الأعمدة) - 1:** المساحة، 2- عدد الغرف، 3- السعر، 4- هل يوجد حديقة.
- **الصفوف (Instances):** (150, 2, 100,000, 0)، (200, 3, 150,000, 1).
- وبالعودة الى الرسمة نعتبر أن x_1 هي سمة و x_2 هي سمة .
- w_1, w_2 أوزان تعبر عن أهمية كل سمة .
- Y المعالج (Processor) : يقوم بمعالجة الإشارات (تُجرى عملية حسابية هي المجموع الموزون الخطي) .

تعتمد عملية اتخاذ القرار داخل البرسبترون على عنصرين رئيسيين:

- **الأوزان (Weights):** لكل إشارة إدخال (x_j) (هناك وزن مقابل لها (w_j)).
- **الوزن:** هو رقم حقيقي يحدد أهمية أو تأثير تلك الإشارة على القرار النهائي. كلما زادت قيمة الوزن، زادت أهمية الإشارة المقابلة له.
- **مثال:** في قرار شراء سيارة، قد يكون وزن (سعر السيارة) أعلى من وزن (لون السيارة).
- **العتبة (Threshold (θ))** هي قيمة رقمية ثابتة. يقوم البرسبترون بحساب مجموع حاصل ضرب كل إشارة في وزنها، ثم يقارن هذا المجموع بالعتبة.

■ سنتعلم خلال الفصل كيفية تحديد قيم الاوزان لاعطاء افضل نتيجة .

$$y = \begin{cases} 0 & W1.X1 + W2.X2 \leq \theta \\ 1 & W1.X1 + W2.X2 > \theta \end{cases}$$

- المخرج (y) : يصدر إشارة واحدة فقط إما 0 أو 1.
- x_1, x_2 : قيم الاشارات الداخلة .
- w_1, w_2 : الأوزان المقابلة .
- إطلاق النيورون (Fires) : إذا تجاوز المجموع الحد (العتبة)، يقال أن النيورون قد انطلق.

تطبيق عملي: محاكاة بوابة AND:

قاعدة بوابة AND: الناتج يكون 1 فقط إذا كانت جميع المدخلات 1

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

لتحقيق هذا لمثال يمكننا اختيار مجموعة لا نهائية من الاوزان و θ سنعطي مبدئيا قيم : $w_1=w_2=0.5$. $\theta = 0.7$

لنختبر هذه القيم :

حالة $x_1 = 1, x_2 = 1$:

المجموع الموزون $(0.5 * 1) + (0.5 * 1) = 1.0$

هل $1 > 0.7$ ؟ نعم اذا الناتج $y = 1$ (يتطابق مع جدول الحقيقة)

نطبق نفس المنطق على الحالات المتبقية. لدينا 2 سمات x_1, x_2 و 4 أمثلة و 1 مخرج



التمثيل البرمجي :

```

2 def AND(x1, x2):
3     w1, w2, theta = 0.5, 0.5, 0.7
4     tmp = x1*w1 + x2*w2
5     if tmp <= theta:
6         return 0
7     elif tmp > theta:
8         return 1
9
10 print("X is : ",AND(0 , 0))
11 print("Y is : ", AND(1 , 0))
12 print("Z is : ", AND(0 , 1))
13 print("S is : ", AND(1 , 1))
14

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
X is : 0
Y is : 0
Z is : 0
S is : 1

```

التحسين الرياضي باستخدام الانحياز (Bias)

لتبسيط التمثيل الرياضي وجعله أكثر مرونة، يتم إعادة صياغة المعادلة عن طريق نقل الحد θ إلى الطرف الأيسر. هذا الرقم المنقول والذي يساوي θ يُشار إليه باسم الانحياز (b) فتصبح المعادلة :

$$y = \begin{cases} 0 & b + W1.X1 + W2.X2 \leq 0 \\ 1 & b + W1.X1 + W2.X2 > 0 \end{cases}$$

حيث : $b = -\theta$

الفرق بين الأوزان (Weights) والانحياز (Bias)

- تخيل أنك مدير شركة (النيورون) وعليك اتخاذ قرار مثل إطلاق منتج جديد. لديك ثلاث مستشارين (المدخلات):
 - مدير التسويق (x_1): يقدم تقرير عن مدى قبول السوق.
 - رئيس قسم الهندسة (x_2): يقدم تقرير عن جودة المنتج.
 - مدير المبيعات (x_3): يقدم تقرير عن توقعات المبيعات.
- الأوزان (w_j): هي مستوى ثقتك بكل مستشار.

وزن كبير (مثل $w > 1$): أنت تثق بهذا المستشار كثيراً. وتعطي تقريره أهمية كبيرة إذا قال نعم فكلامه مؤثر بشكل كبير على قرارك.

وزن صفر (مثل $0 < w < 1$): أنت تثق به بشكل معتدل.

وزن سالب (مثل $w < 0$): هذا هو الأهم. أنت لا تثق بهذا المستشار بل تعتقد أن رأيه غالباً ما يكون عكس الصواب. إذا قال نعم، فكلامه يجعلك تميل إلى قول "لا".

■ الخلاصة: الأوزان هي نوع من المعلومات التي تتحكم في أهمية كل إشارة على حدى، هي تخص كل مدخل بشكل منفصل.

■ الانحياز (b): ليس متعلقاً بأي من المستشارين. إنه ميلك الطبيعي كصانع قرار حتى قبل أن تستمع إلى أي منهم.

انحياز موجب كبير (مثلاً $b > 0$): أنت شخص متفائل ومتحمس .

لديك ميل طبيعي لاتخاذ القرار (نعم) (إطلاق المنتج)، حتى لو كانت تقارير المستشارين ضعيفة قد تقرر إطلاق المنتج لإنك في مزاج جيد

انحياز سالب كبير (مثلاً $b < 0$): أنت شخص محافظ وحذر جداً .

لديك ميل طبيعي لاتخاذ القرار (لا) تحتاج إلى تقارير إيجابية وقوية جداً ومقنعة من جميع المستشارين حتى تغير رأيك وتوافق على الإطلاق.

■ الخلاصة: الانحياز هو معلمة تتحكم في سهولة تنشيط النيورون بشكل عام، بغض النظر عن المدخلات. أي هو لا يخص مدخل معين بل يخص النيورون ككل.

جدول مقارنة بين الأوزان والانحياز:

وجه المقارنة	الأوزان	الانحياز
التعريف	معامل الضرب لكل إشارة إدخال	قيمة ثابتة تضاف إلى المجموع
الغرض	التحكم في أهمية كل مدخل على حدى	التحكم في سهولة تنشيط النيورون بشكل عام
التأثير	يغير شكل دالة التنشيط	يزيح دالة التنشيط يميناً أو يساراً
الكمية	لكل نيورون يوجد عدد من الأوزان يساوي عدد المدخلات	لكل نيورون يوجد انحياز واحد فقط

```

2 import numpy as np
3
4 x = np.array([0, 1]) # Input
5 w = np.array([0.5, 0.5]) # Weight
6 b = -0.7 # Bias
7 s=w*x
8 print("S is : " , s)
9 z = np.sum(s)
10 print("Z is : ", z)
11 j= z + b
12 print("J is : ",j)
13

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
S is : [0.  0.5]
Z is : 0.5
J is : -0.19999999999999996

```

الشرح :

- المدخل x ب 2 سمات هي array ببعد واحد .
- الوزن (w): هي ايضا vector يجب أن يكون عدد الأوزان = عدد المدخلات، لأنه سيتم إجراء عملية ضرب بين كل مدخل * وزنه المقابل.
- المجموع (Sum): نرسل له array ويقوم بجمع العناصر بغض النظر عن البعد .
- إذا تم ضرب كل مدخل بما يقابله من وزن ثم جمع النواتج ثم إضافة قيمة الانحياز.

التنفيذ باستخدام الأوزان والانحياز :

يمكننا استخدام كل من الانحياز والأوزان لتنفيذ بوابة AND

```

2 import numpy as np
3
4 def AND(x1, x2):
5     x = np.array([x1, x2])
6     w = np.array([0.5, 0.5])
7     b = -0.7
8     tmp = np.sum(w*x) + b
9     if tmp <= 0:
10         return 0
11     else:
12         return 1
13
14 print(AND(0,0))
15 print(AND(1,1))
16 print(AND(1,0))
17 print(AND(0,1))
18

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
0
1
0
0

```

- ملاحظة: الانحياز يعمل بشكل مختلف عن الأوزان. الانحياز يتحكم في مدى سهولة الحصول على مخرج يساوي 1 بغض النظر عن المدخلات.
- إذا كانت $b = -0.1$ (انحياز سالب صغير)، تطلق الخلية العصبية عندما يتجاوز المجموع الموزون 0.1.
- إذا كانت $b = -20$ (انحياز سالب كبير)، تطلق الخلية العصبية عندما يتجاوز المجموع الموزون 20.
- عندما يكون الانحياز سالب كبير يعني ان النيورون كسول ويحتاج الى مجموع موزون كبير جدا لينطلق
- وعندما يكون سالب صغير هذا يعني ان النيورون متحمس ويمكن ان ينطلق حتى مع مدخلات صغيرة
- تسمية المعاملات: على الرغم من أن w_j و b تطلق عليهما أوزان وانحياز، فإن جميع المعاملات w_j , b تسمى أوزاناً حسب السياق.

ملخص عمل البرسبترون :

- تعميم القانون من أجل زمدخل و زوزن (يجب أن يكون عدد الأوزان مساوياً لعدد المدخلات).

$$\sum w_j X_j$$

يتم ضرب كل مدخل بالوزن المقابل له ثم جمع النواتج ويسمى المجموع الموزون

- يتم مقارنة ناتج المجموع الموزون مع العتبة ، والمخرج يكون إما 0 أو 1.

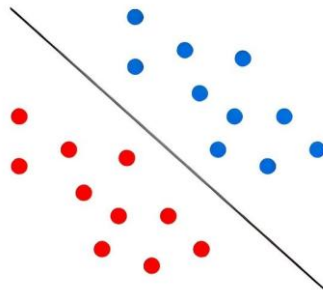
فتصبح المعادلة على الشكل التالي وهو الشكل الذي يهمنا والذي سوف نتعامل معه:

$$f = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

- يتم إسناد المتغير $w \cdot x + b$ إلى z فتصبح :

$$f = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$

- إن z تشبه معادلة خط مستقيم فإن الدالة التنشيط تنشئ مستوى يقسم توزيع الإدخال إلى فئتين متساويتين لتفصل بين البيانات كما هو موضح في الشكل :



■ إن البرسيبترون أحادي الطبقة عاجز عن حل المشكلات غير الخطية لأنه لا يستطيع إلا رسم خط مستقيم للفصل بين البيانات، مثل بوابة XOR سنتحدث عنها لاحقا بالتفصيل هذا القيد هو الذي أدى الى تطور البرسيبترون متعدد الطبقات وهو الاساس الذي بنيت عليه الشبكات العصبية الحديثة

مشكلة XOR

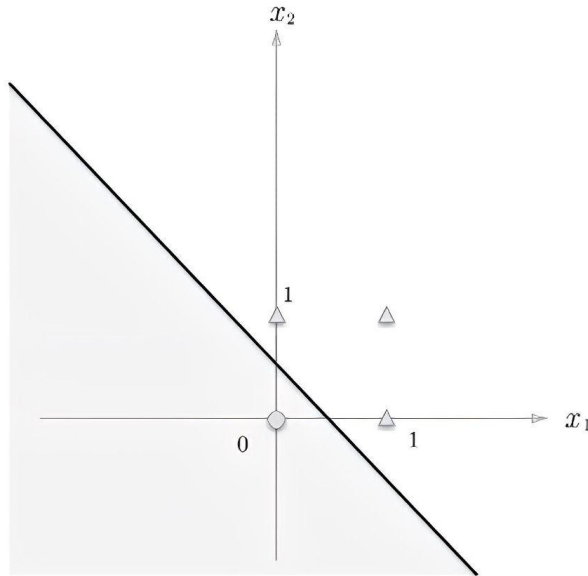
- بعد النجاح في محاكاة بوابات AND و OR، يظهر تحدي أكبر وهو محاكاة بوابة XOR.
- قاعدة هذه البوابة: الناتج يكون 1 فقط إذا كان أحد المدخلات 1 وليس كلاهما.

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

- النتيجة: من المستحيل بناء بوابة XOR باستخدام برسيبترون أحادي الطبقة. لا توجد مجموعة من الأوزان والانحياز يمكنها تحقيق جدول الحقيقة هذا.
- التفسير الهندسي: الخطوط والمساحات النقاط التي ناتجها 0 يمكن تمثيلها ب دوائر والنقاط التي ناتجها 1 يمكن تمثيلها ب مثلثات
- ف البرسيبترون أحادي الطبقة، من خلال معادلته الخطية، يقوم برسم خط مستقيم واحد يقسم المستوى إلى منطقتين. منطقة واحدة يعطى لها الخرج 1 والاخرى المخرج 0
- في حالة AND و OR، يمكن رسم خط مستقيم واحد يفصل (الدوائر) عن (المثلثات). هذه المشاكل تسمى قابلة للفصل الخطي.
- لنحاكي بوابة OR على سبيل المثال : $w_1 = 1, w_2 = 1, b = -0.5$

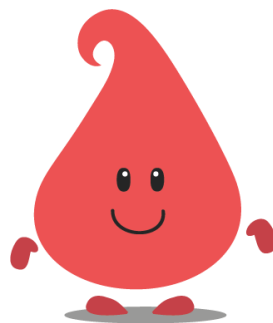
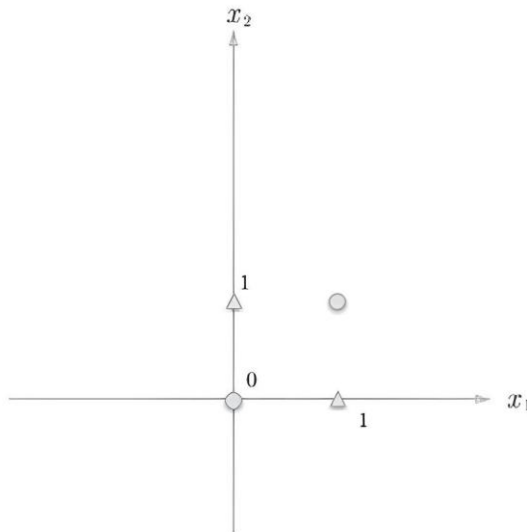
$$y = \begin{cases} 0 & -0.5 + x_1 + x_2 \leq 0 \\ 1 & -0.5 + x_1 + x_2 > 0 \end{cases}$$

ولدينا منطقتين منطقة يعطى لها الخرج 1 والأخرى 0 كما في الشكل :



أما في حالة XOR (حيث الدوائر والمثلثات هي مخرجات بوابة XOR)، فإنه مهما حاولنا، لن نستطيع الفصل بين الدوائر والمثلثات بخط مستقيم، لأنه لا يمكن لخط مستقيم واحد أن يفصل بينها.

مثال :



مفهوم الخطية وعدم الخطية

أولاً: الخطية (Linearity)

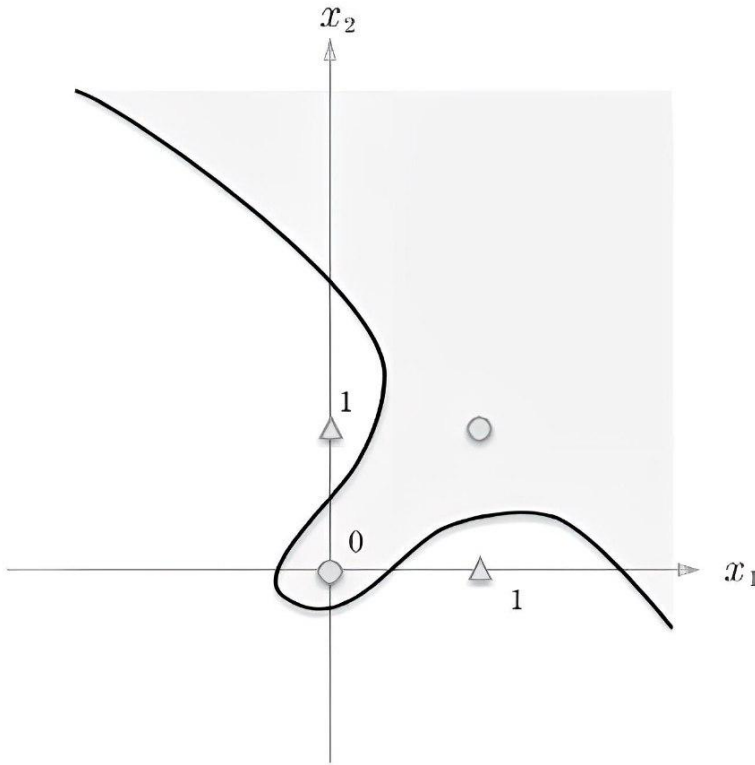
ببساطة: هي القدرة على فصل البيانات باستخدام خط مستقيم واحد.

النظام الخطي: هو نظام يستطيع رسم خط مستقيم واحد على الرسم البياني، حيث يفصل هذا الخط بين فئتين مختلفتين من البيانات كل النقاط على يمين الخط تنتمي لفئة مثلاً الناتج 1 و كل النقاط على يساره تنتمي للفئة الأخرى الناتج 0
مثال: بوابة OR وبوابة AND.

ثانياً: عدم الخطية (Non-Linearity)

عدم القدرة على فصل البيانات باستخدام خط مستقيم واحد. يحتاج إلى منحنيات أو أشكال معقدة للفصل.

النقاط من الفئات المختلفة تكون مختلطة معاً بطريقة تتطلب حدوداً منحنية للفصل بينها كما في الرسم البياني التالي للبوابة XOR.



جدول مقارنة: الخطية وعدم الخطية :

وجه المقارنة	الخطية	عدم الخطية
التعريف	يمكن الفصل بخط مستقيم	لا يمكن الفصل يحتاج إلى منحنيات
مثال	بوابات OR و AND	بوابة XOR
النموذج القادر على حلها	البرسيبترون أحادي الطبقة	البرسيبترون متعدد الطبقات
دالة التنشيط	دالة الخطوة (step function)	دالة غير خطية مثل sigmoid
القوة	محدودة لا يمكنه حل مشاكل حقيقية معقدة	قوي جداً أساس التعلم العميق والذكاء الاصطناعي الحديث

البرسبترون متعدد الطبقات (Multilayer Perceptron - MLP)

حل مشكلة XOR

الفكرة الجوهرية : التكامل عبر الطبقات

حل مشكلة XOR لا يكمن في تعقيد النيورون الواحد بل في دمج عدة نيورونات بسيطة في هيكل طبقي ف الفكرة هي تقسيم المشكلة المعقدة (XOR) إلى مشاكل فرعية بسيطة مثل (AND, OR, NAND) يمكن للبرسبترون حلها، ثم دمج مخرجات هذه المشاكل للحصول على الحل النهائي.

حل XOR باستخدام الطبقات:

يمكن بناء بوابة XOR من خلال دمج ثلاث بوابات أساسية NAND , OR , And



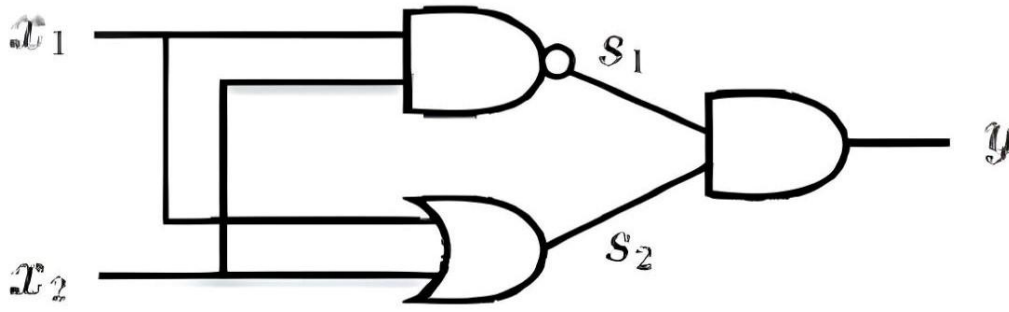
OR



AND



NAND



الميكمل الشبكي :

الطبقة الأولى (Hidden Layer) :

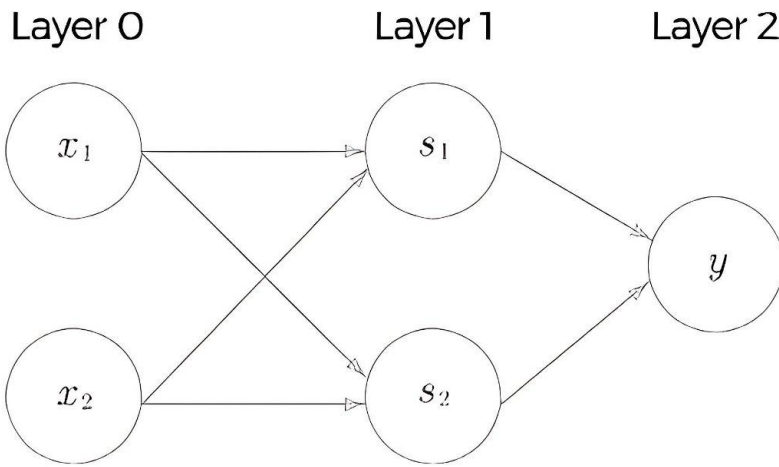
نيورون 1 ينفذ عملية NAND على المدخلات الأصلية.

نيورون 2 ينفذ عملية OR على المدخلات الأصلية.

الطبقة الثانية (Output Layer) :

نيورون 3 يستقبل مخرجات الطبقة الأولى (مخرج NAND ومخرج OR) وينفذ عليهما عملية AND

المخرج النهائي لهذا النيورون هو الخرج XOR.



التحقق من الحل :

لنفترض أن مخرج NAND هو s1 ومخرج OR هو s2 يمكننا بناء جدول الحقيقة :



x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

عندما ننظر إلى الأعمدة x_1, x_2, y نجد أنها تتطابق مع جدول الحقيقة للبوابة XOR

```

1  # AND GATE
2  def AND(x1, x2):
3      w1, w2, theta = 0.5, 0.5, 0.7
4      tmp = x1*w1 + x2*w2
5      if tmp <= theta:
6          return 0
7      elif tmp > theta:
8          return 1
9  # OR GATE
10 def OR(x1, x2):
11     w1, w2, theta = 0.5, 0.5, 0.3
12     tmp = x1*w1 + x2*w2
13     if tmp >= theta:
14         return 1
15     elif tmp < theta:
16         return 0
17 # NAND GATE
18 def NAND(x1, x2):
19     w1, w2, theta = 0.5, 0.5, 0.7
20     tmp = x1*w1 + x2*w2
21     if tmp <= theta:
22         return 1
23     elif tmp > theta:
24         return 0
25 # XOR GATE
26 def XOR ( x1 , x2 ):
27     s1 = NAND ( x1 , x2 )
28     s2 = OR ( x1 , x2 )
29     y = AND ( s1 , s2 )
30     return y
31 # TEST
32 print(XOR ( 0 , 0 ) )
33 print(XOR ( 1 , 0 ))
34 print(XOR ( 0 , 1 ) )
35 print(XOR ( 1 , 1 ) )

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
0
1
1
0

```

البنية الشبكية والطبقات :

هذا الهيكل يمثل شبكة عصبية بسيطة على الرغم من وجود 3 اعمدة من النيورونات الا انه تسمى هذه الشبكة شبكة من طبقتين (Two-Layer Network) لأن هناك طبقتين فقط تحتويان على أوزان (طبقة بين 0 و 1 و طبقة بين 1 و 2).

القوة: ما لا يمكن تحقيقه بنيورون واحد، يمكن تحقيقه بواسطة عدة نيورونات في طبقات هذا المبدأ هو اساس قوة الشبكات العصبية العميقة .

التفصيل الهيكلي والوظيفي

البرسيبترون أحادي الطبقة:

الهيكليّة:

- يتكون من طبقة واحدة فقط من الأوزان القابلة للتعليم.
- المدخلات متصلة مباشرة الى طبقة المخرجات. لا توجد طبقات وسيطة (مخفية).
- دالة التنشيط: يستخدم حصرية دالة الخطوة (Step Function)، الناتج إما 0 أو 1.

القدرات والقيود :

- نقطة القوة: ممتاز في حل المشكلات الخطية القابلة للفصل مثل بوابات OR و AND.
- نقطة الضعف: عاجز تماماً عن حل المشاكل غير الخطية مثل مشكلة XOR هذا القيد الذي ادى الى ظهور النوع متعدد الطبقات
- مثال: قاضي واحد يُقدّم له كل الأدلة (المدخلات) مرة واحدة فيصدر حكماً نهائياً (مخرج 0 أو 1) استناداً إلى قاعدة بسيطة.

البرسيبترون متعدد الطبقات:

الهيكليّة:

- تتكون من طبقة إدخال، وطبقة أو أكثر من الطبقات المخفية، وطبقة إخراج.
- الأوزان: الأوزان موجودة بين كل طبقة والتي تليها.
- دالة التنشيط: يجب أن تستخدم دوال تنشيط غير خطية مثل Sigmoid أو ReLU. (هذا هو الشرط الأساسي لقدرة الشبكة على حل المشاكل غير الخطية).

القدرات والقيود :

- نقطة القوة: قادرة على حل المشاكل غير الخطية المعقدة مثل XOR، والتعرف على الصور والترجمة الآلية (الأساس الذي بني عليه التعلم العميق).
- نقطة الضعف: أكثر تعقيداً في البناء والتدريب، ويتطلب خوارزمية الانتشار الخلفي (Backpropagation).
- مثال: لجنة من الخبراء تعمل بشكل هرمي :
الطبقة المخفية الأولى: مجموعة من الخبراء المبتدئين، كل خبير يبحث عن ميزة بسيطة (هل يوجد خط مستقيم؟).

الطبقة المخفية الثانية: مجموعة من الخبراء المتوسطين، يأخذون تحليلات الخبراء المبتدئين، ويبحثون عن ميزات أكثر تعقيداً (هل يوجد عين؟).
طبقة الإخراج: الخبير الأعلى يجمع كل التحليلات ويصدر الحكم النهائي (هل هذه صورة وجه؟).

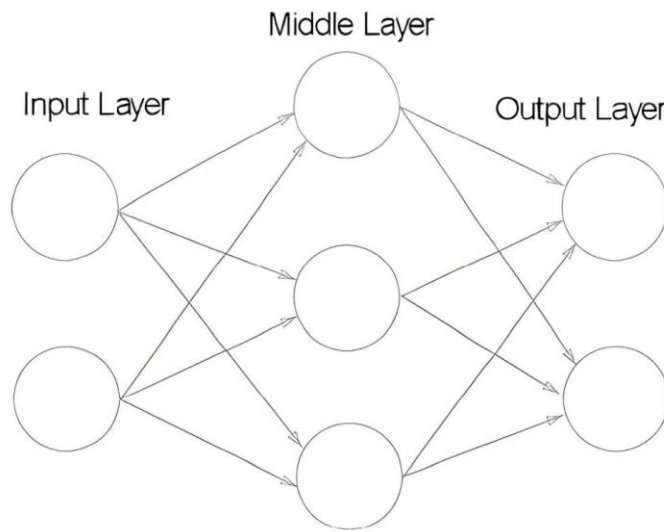
مقارنة بين البرسيبترون أحادي ومتعدد الطبقات :

وجه المقارنة	أحادي الطبقة	متعدد الطبقات
البنية	طبقة أوزان واحدة	طبقات متعددة (مدخل , مخفية , مخرج)
دالة التنشيط	دالة الخطوة (step function)	دالة غير خطية (sigmoid , Relu)
قدرة حل المشاكل	خطية فقط AND و OR	خطية وغير خطية XOR

من البرسيبترون إلى الشبكات العصبية neural network

ماذا تغير ؟

البنية الطبقة للشبكات العصبية هي نفس بنية البرسيبترون متعدد الطبقات الفارق الجوهري والوحيد هي دالة التنشيط Activation function المستخدمة داخل كل نيورون

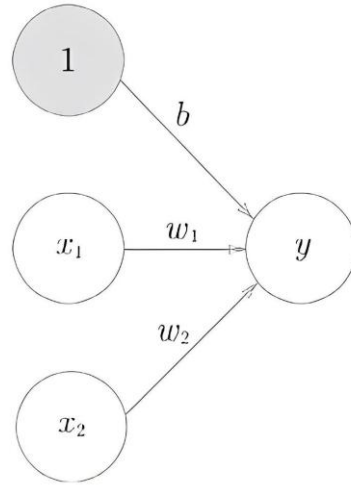


الطبقة الأولى : تسمى طبقة الإدخال input layer .

الطبقة الثانية : الطبقة الوسطى أو المخفية hidden layer تسمى مخفية لأننا لا نتعامل مع مخرجاتها مباشرة بل تعمل في الداخل .

الطبقة الأخيرة : طبقة الإخراج output layer .

تمثيل Bias



في الشبكات العصبية يتم التعامل مع الانحياز كأنه نيورون إضافي كيف؟

- نتخيل أن هناك نيورون إضافي في كل طبقة مدخلة دائماً هو القيمة الثابتة 1 .
 - لهذا النيورون الخاص وزن واحد فقط يربطه بالنيورونات في الطبقة التالية هذا الوزن هو في الحقيقة b قيمة الانحياز .
 - لماذا هذا التمثيل مهم؟
- لأنه يوحد المعادلة الآن كل نيورون في الطبقة التالية يستقبل إشارات من جميع نيورونات الطبقة السابقة بما في ذلك نيورون الانحياز ويضرب كل إشارة في وزنها ويجمعها كلها

$$y = \begin{cases} \rightarrow 0 & b + W1.X1 + W2.X2 \leq 0 \\ \rightarrow 1 & b + W1.X1 + W2.X2 > 0 \end{cases}$$

نرمز للدخل ب X

وللتبسيط نستخدم التابع h فيصبح :

$$y = h(b + W1.X1 + W2.X2)$$

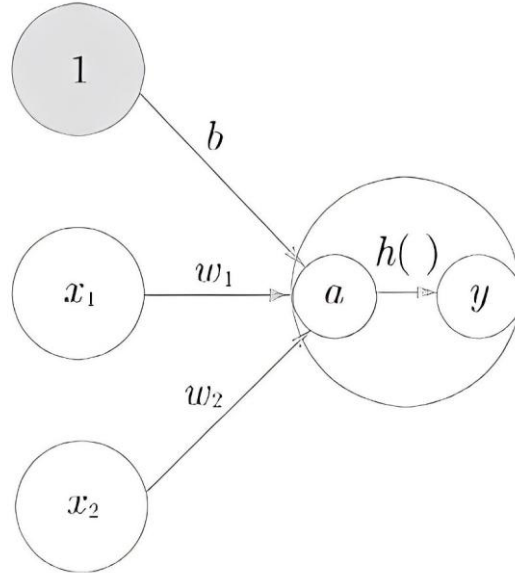
التابع h يأخذ المجموع الموزون

$$y = h(x)$$

$$h(x) = \begin{cases} 0 & x \leq 0 \\ 1 & x > 0 \end{cases}$$

دالة التنشيط (Activation function)

- هنا يكون التغيير الجوهرى والأهم في البرسيبترون البسيط كنا نستخدم دالة حادة أما في الشبكات العصبية نستخدم دوال أكثر مرونة .



إدخال دالة التنشيط $h(x)$:

- بعد أن يقوم النيورون بحساب المجموعة الموزون (لنسميه a) فإنه لا يطلق مباشرة كنتاج نهائي بل يمرره أولاً على دالة تنشيط $h(x)$.

دور دالة التنشيط:

- هي التي تحدد كيف سيستجيب النيورون للمجموعة الموزون هي شخصية النيورون وطريقة تفكيره .
- الفرق الحاسم بين البرسيبترون البسيط مقابل الشبكة العصبية :
- البرسيبترون البسيط هو شبكة أحادية الطبقة تستخدم داله الخطوة (step function) كدالة تنشيط هي حاده وثنائيه تشبه الدرجة.
- البرسيبترون متعدد الطبقات هو ما نسميه بالشبكة العصبية وهو يستخدم دالة تنشيط ناعمة (smooth) مثل دالة (sigmoid).

أنواع دالة التنشيط :

1. دالة الخطوة (step function) :

- المعادلة $h(x) = 1$ إذا $x > 0$ و $h(x) = 0$ خلاف ذلك
- السلوك : تصدر مخرجاً ثنائياً (0 أو 1) الانتقال من 0 إلى 1 يحدث بشكل فجائي عند الصفر .
- العيوب : طبيعتها الفجائية والغير قابلة للتفاضل تجعلها غير مناسبة لعمليات التعلم المعقدة في الشبكات متعددة الطبقات .

```

2 import numpy as np
3
4 def step_function(x):
5     y = x > 0
6     return y.astype(np.int64)
7
8 print(step_function(np.array([1,2,3])))
9

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\vs projects\python projects> python test.py
[1 1 1]

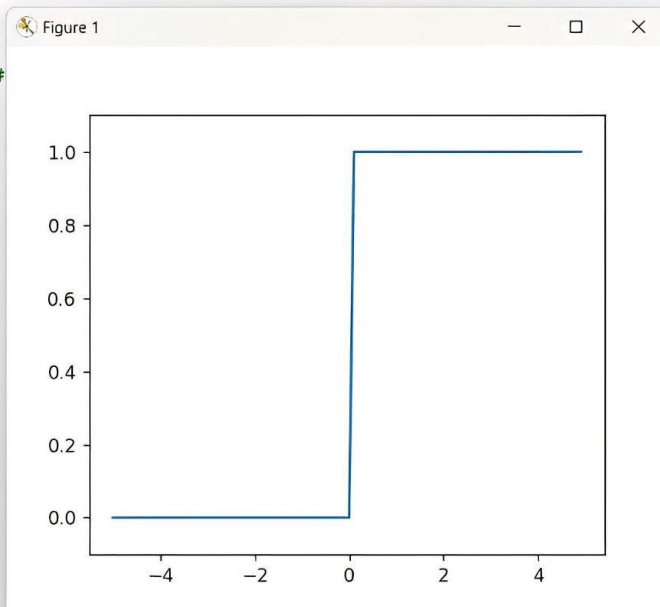
```

الشرح : عندما نقارن المصفوفة Numpy والرقم صفر فإن Numby تقوم بالمقارنة لكل عنصر على حدى وتنتج مصفوفة من القيم المنطقية (true/false) فنستخدم الدالة astype لتحويل هذه القيم المنطقية إلى أرقام int فتصبح $0 \rightarrow \text{false}$, $1 \rightarrow \text{true}$.

```

2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def step_function(x):
6     return np.array(x > 0, dtype=np.int64)
7
8 x = np.arange(-5.0, 5.0, 0.1)
9 y = step_function(x)
10 plt.plot(x, y)
11 plt.ylim(-0.1, 1.1) #
12 plt.show()
13

```



الشرح :

دالة الخطوة التي تأخذ المتغير x الذي يمثل المجموع الموزون
 هذه الخطوة تحول المصفوفة المنطقية الناتجة إلى مصفوفة من الأرقام الصحيحة
 True تصبح 1, false تصبح 0.

`x=np.arange` إنشاء مصفوفة `x` تحتوي على جميع قيم الأرقام من 5- إلى 4.9 بزيادة قدرها 0.1 هذه المصفوفة تمثل المدخلات التي سنمررها لدالة الخطوة لغرض الرسم البياني .
`y` : تطبق الدالة التي عرفناها على جميع المصفوفة `x` هذا يعطينا المصفوفة `y` التي تحتوي فقط القيم 0 و 1 .

`plt.plot` : دالة الرسم الرئيسية التي تأخذ مصفوفة الإحداثيات الأفقية `x` ومصفوفة الإحداثيات العمودية `y` ليكون من -0.1 إلى 1.1 فقط
 - هذا الضمان أن القيم 0 و 1 (التي هي المخرجات الوحيدة) تكون مرئية بوضوح .
`plt.show` : تعرض الرسم البياني .

دالة الخطوة تدعى أحياناً دالة السلم

2. sigmoid function :

$$h(x) = \frac{1}{1+e^{-x}}$$

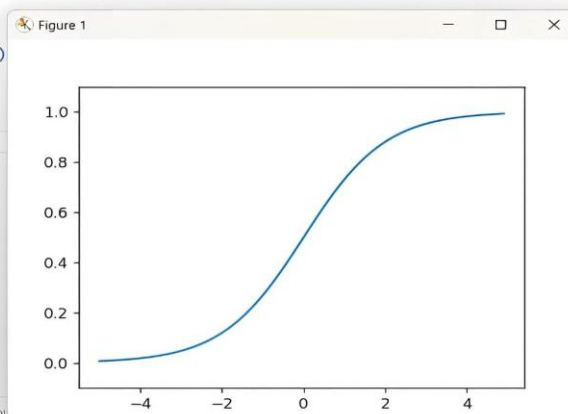
- المعادلة :
- السلوك : دالة ناعمة ومستمرة المخرج ليس مجرد 0 أو 1 ب هو أي رقم حقيقي في النطاق (0,1)
 أي مثلاً 0.001 أو 0.99
- الأهمية : هذه النعومة والاستمرارية تسمح بإجراء تعديلات دقيقة ومتدرجة على الأوزان أثناء عملية التدريب
- أمثلة:

```
2 import numpy as np
3
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6 x = np.array([-1.0, 1.0, 2.0])
7 print(sigmoid(x))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\vs projects\python projects> python test.py
[0.26894142 0.73105858 0.88079708]
```

```
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def sigmoid(x):
6     return 1 / (1 + np.exp(-x))
7
8 x = np.arange(-5.0, 5.0, 0.1)
9 y = sigmoid(x)
10 plt.plot(x, y)
11 plt.ylim(-0.1, 1.1)
12 plt.show()
13
14
15
16
17
18
```



PROBLEMS OUTPUT DEBUG CONSOLE

مقارنة بين sigmoid و دالة الخطوة :

دالة sigmoid هي منحن سلس حيث يتغير الناتج باستمرار بناءً على المدخلات أما دالة الخطوة يتغير الخرج فجأة عند الصفر .

من حيث النعومة (smoothness) :

Sigmoid ناعمة ومستمرة بينما دالة الخطوة حادة و فجائية هذه النعومة ضرورية لعملية التعلم في الشبكات العصبية.

طبيعة المخرجات :

دالة الخطوة تخرج فقط 1 أو 0 (إشارات ثنائية) أما دالة sigmoid تخرج أرقام حقيقية مستمرة بين 0 و 1 يعني أن الشبكات العصبية تتعامل مع إشارات مستمرة وليست ثنائية فقط .

دالة الخطوة تشبه أنبوب يصدر صوت حاد عند امتلائه أما sigmoid يشبه عجلة ماء تدور بسلاسة بناءً على كمية الماء التي تصل إليها .

يمكننا ملاحظة أن الدالتان تختلفان في النعومة وتتشابهان في الشكل تخرج كلتاها قيمة قريبة من الصفر عندما يكون المدخل صغير ومع ازدياد المدخل يقترب المخرج من الواحد , تخرج الدالتان قيمة كبيرة عندما تحتوي إشارة المدخل على معلومات مهمة و تخرجان قيمة صغيرة عندما لا تحتوي على هذه المعلومات كما تتشابهان في أنهما تخرجان قيم بين 0 و 1 .

دالة الاخطية :

- غالباً ما يستخدم مصطلح دالة غير خطية ودالة خطية عند استخدام دالة التنشيط
- الدالة هي محول يرجع قيمة عند إدخالها
- تسمى الدالة التي تخرج قيم المدخلات مضروبة في ثابت بالدالة الخطية $h(x) = cx$ حيث c ثابت لذلك يكون رسم الدالة الخطية خط مستقيم بالمقابل فإن رسم الدالة غير الخطية ليس خط مستقيم بسيط .
- دالة الخطوة و sigmoid متشابهتان من ناحية أخرى ومن أهم أوجه التشابه أنهما دالتان غير خطيتان تمثل دالة sigmoid بمنحني بينما دالة الخطوة تمثل بخطوط مستقيمة تشبه السلالم

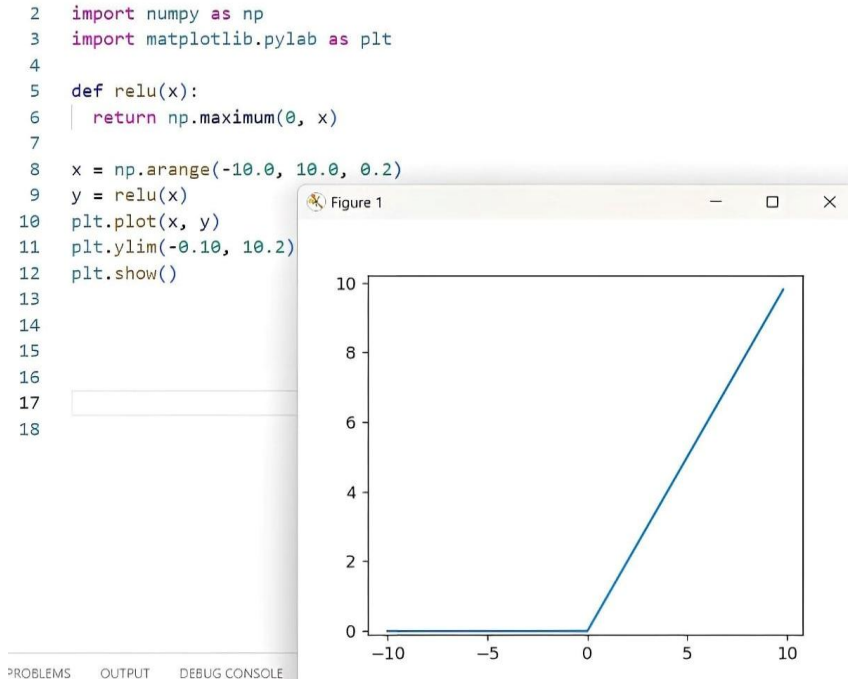
لماذا يجب أن تكون دالة التنشيط غير خطية ؟

هذا هو المبدأ الأساسي .

- المشكلة أنه إذا استخدمنا دالة تنشيط خطية فإن تكديس الطبقات يصبح بلا فائدة .
 - إذا كانت $h(x) = cx$ وكانت لدينا شبكة من 3 طبقات فستكون المعادلة :
- $$y = h(h(h(x))) = c \times (c \times (c \times x)) = c^3 \times x$$
- هذه الشبكة المعقدة ذات الثلاث طبقات تتصرف تماماً مثل شبكة من طبقة واحدة لها وزن واحد فقط من دون طبقة مخفية $a = c^3$ لقد فقدنا كل فائدة التعمق
 - إذاً الاخطوية تمنح كل طبقة هويتها الخاصة وقدرتها على تعلم شيء جديد وهي التي تمنح الشبكة بأكملها قدرة هائلة .

3. دالة Relu : (rectified linear unit)

- المعادلة : $h(x) = \max(0, x)$
- السلوك : إذا كان المدخل موجب فإنها تخرجه كما هو أما إذا كان المدخل صفر أو سالب فإنها تخرج صفر
- الأهمية : هي دالة التنشيط الأكثر استخداماً اليوم في الشبكات العصبونية لسببين رئيسيين :
بساطتها الحسابية (سريعة جداً).
- فعاليتها في حل مشكلة تلاشي التدرج vanishing gradient التي يمكن أن تحدث مع دالة sigmoid .
- التمثيل البرمجي :



مقارنة بين البرسيبترون و NN :

1. الفرق في البنية :

- البرسيبترون: هو كيان منفرد لديه مجموعة من المدخلات وينتج مخرج واحد (لا يوجد طبقات وسيطة/ مخفية) .
- أما الشبكة العصبونية : هي هيكل هرمي أو طبقي .
طبقة الإدخال التي تستقبل البيانات ثم الطبقات المخفية التي تشكل عقل الشبكة حيث كل طبقة مخفية تأخذ مخرجات الطبقة التي تسبقها وتعالجها لاستخلاص ميزات أكثر تعقيداً .
ثم طبقة الإخراج التي تنتج النتيجة النهائية .

2. الفرق في القدرة :

- البرسيبترون: فشل في حل XOR لأنه لا يستطيع رسم منحنى هو قادر على الفصل الخطي فقط .
- الشبكة العصبونية (برسيبترون متعدد الطبقات) : نجحت في حل XOR لأن الطبقات الأولى قامت بتحويل المشكلة غير الخطية إلى مشكلة خطية يمكن للطبقة الأخيرة حلها .

- القوة ليست في النيورون نفسه بل في الترتيب الطبقي الذي يسمح باستخلاص ميزات هرمية .

3. الفرق التقني (دالة التنشيط):

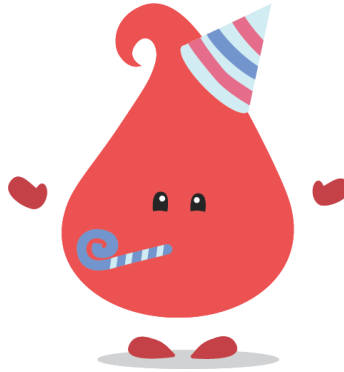
- هذا هو الفرق الذي يمكّن للشبكات العصبية من تحقيق قوتها .
- دالة الخطوة : حادة وفجائية لا تسمح بتعديلات دقيقة .
- دوال مثل Relu , sigmoid , ناعمة وقابلة للتفاضل وضرورية جداً لعملية التعلم التلقائي , هي تسمح للخوارزمية بأن تقول هذا الوزن ساهم في الخطأ بمقدار x لذا يجب أن نعدله قليلاً بهذا الاتجاه مع دالة الخطوة لا يمكنك أن تعرف مدى الخطأ فقط إذا كان هناك خطأ أم لا .

4. طريقة التعلم :

- في البرسيبترون كما تحدثنا عنه نحن المهندسين فكرنا في المشكلة (AND gate), وجربنا أوزان مختلفة يدوياً حتى وجدنا الحل هذه العملية غير قابلة للتطبيق على مشاكل حقيقية .
- أما في الشبكة العصبية : نحن المدربين نحدد الأوزان , نعطي الشبكة آلاف الأمثلة (البيانات) والنتائج الصحيحة ونتركها هي تكتشف الأوزان المثلى بنفسها عبر عملية تكرارية من التنبؤ وحساب الخطأ والتعديل.

وجه المقارنة	البرسيبترون	الشبكة العصبية
التعريف	نيورون اصطناعي واحد	شبكة متكاملة من نيورونات اصطناعية متعددة
البنية الهيكلية	طبقة واحدة (مدخلات - معالج - مخرج)	طبقات متعددة (طبقة إدخال - طبقة أو طبقات مخفية - طبقة إخراج)
حل المشاكل	قادرة فقط على حل المشاكل الخطية مثل AND, OR	قادرة على حل المشاكل الخطية وغير الخطية المعقدة مثل التعرف على الصور وترجمة اللغات وبوابة XOR
دالة التنشيط	نستخدم دالة الخطوة وهي دالة غير ناعمة	تستخدم دوال تنشيط ناعمة غير خطية مثل RelU, sigmoid
التعلم	الأوزان تحدد يدوياً	الأوزان تتعلم تلقائياً من البيانات عبر عملية التدريب وخوارزمية الانتشار الخلفي

- مثال : البرسيبترون هو آلة حاسبة بسيطة تعطيها الأرقام وتعطيك الناتج أما الشبكة العصبية هي عالم رياضيات تعطيه كتاباً مليئاً بالمسائل وحلولها وبعد فترة يصبح قادراً على حل مسائل جديدة لم يرها من قبل .



مقارنة سريعة بين أنواع دوال التنشيط :

1. دالة الخطوة :

تنشيه : مثل مفتاح كهرباء إما أن يكون مفتوح بالكامل (1) أو مغلق تماماً (0) لا يوجد شيء بينهما .

الميزات :

- سهولة الفهم والتنفيذ .
- واضحة تعطي تكراراً ثنائياً حاد وهو ما نريده في المسائل المنطقية البسيطة .

العيوب :

- غير ناعمة : الانتقال من 0 إلى 1 يحدث بشكل فجائي عند الصفر وهذا يجعلها غير قابلة للتفاضل عند هذه النقطة .
- مشكلة التعلم : عملية تعلم الشبكة العصبية تعتمد على حساب ميل الدالة لتحديد كيفية تعديل الأوزان , بما أن ميل دالة الخطوة هو صفر في كل مكان ما عدا عند الصفر (حيث يكون غير معرف) فإن الخوارزمية لا تعرف في أي اتجاه يجب أن تعدل الأوزان إنها تقتل عملية التعلم .
- فقدان المعلومات : هي تقول فقط نشط أو غير نشط هي لا تقول مدى نشاطه هذه المعلومات الدقيقة ضرورية للتعلم .

الأهمية للشبكة العصبية :

محدودة جداً هي مناسبة للبرسيبترون البسيط لكنها غير عملية على الإطلاق للشبكات العصبية العميقة لأنها تمنعها من التعلم .

2. دالة sigmoid

تنتج قيم ناعمة ومستمرة بين 0 و 1

تشبيه :

مثل مغير شدة الإضاءة يمكنك التحكم في الإضاءة بشكل تدريجي من الظلام التام (0) إلى الإضاءة الكاملة (1).

الميزات :

- ناعمة و قابلة للتفاضل : ميلها محدد في كل نقطة مما يسمح لخوارزمية الانتشار الخلفي بعمل حسابات دقيقة وتعديل الاوزان بشكل صريح .
- مخرجات محدودة (0 إلى 1) هذا مفيد جداً خاصة في طبقة الإخراج للمشاكل الثنائية (مثل: هل هذا بريد مزعج) حيث يمكن تفسير المخرج على أنه احتمالية .

العيوب:

- مشكلة تلاشي التدرج : عندما يكون المدخل x كبير جداً أو سالباً جداً يصبح منحنى الدالة مسطحة جداً (ميلها قريب من الصفر) في الشبكات العصبية يتم ضرب هذه الميول الصغيرة جداً معاً أثناء الانتشار الخلفي مما يجعل الإشارة تتلاشى قبل أن تصل إلى الطبقات الأولى فالنتيجة : الطبقات الأولى لا تعلم شيئاً .
- التكلفة الحسابية : عملية حساب الأس أثقل حسابياً من العمليات البسيطة

الأهمية للشبكة العصبية :

تاريخية ومهمة لفترة طويلة كانت هي الخيار الافتراضي لا تزال مفيدة في طبقة الإخراج لمشاكل التصنيف الثاني لكنها نادراً ما تستخدم في الطبقات المخفية للشبكات العصبية بسبب مشكلة تلاشي التدرج

3. دالة ReLU :

تشبيه :

بوابة إيجابية تسمح لكل الإشارات الإيجابية بالمرور كما هي وتحجب كل الإشارات السالبة تماماً

الميزات :

- كفاءة حسابية عالية : عملية $\max(0, x)$ سريعة جداً وبسيطة للغاية للحاسوب مقارنة ب الأس في sigmoid هذا يسرع عملية التدريب بشكل كبير .
- تحل مشكلة تلاشي التدرج (جزئياً) للمدخلات الموجبة , ميل الدالة ثابت ويساوي 1 . هذا يعني أن إشارة التعلم يمكن أن تمر عبر النيورونات النشطة دون أن تضعف . مما يسمح للشبكة العميقة بالتعلم بفعالية .
- أي إدخال سالب سيؤدي إلى مخرج يساوي الصفر هذه يعني أن بعض النيورونات ستكون خامدة في أي لحظة مما يجعل الشبكة أكثر كفاءة واقل عرضة للتعلم الزائد.

العيوب :

- Dying ReLU : إذا كانت أوزان نيورون تتحدث بشكل مستمر بحيث يصبح مدخله سالباً دائماً فإن مخرجه سيكون دائماً صفر وميله سيكون صفر سيتوقف هذا النيورون عن التعلم تماماً وكأنه مات .

المخرجات ليست متمركزة حول الصفر : جميع المخرجات غير سالبة مما قد يسبب بعض المشاكل الدقيقة في تدفق التدرجات أثناء التدريب

الأهمية في الشبكة العصبية :

هي الخيار الأفضل والمعياري الحالي للطبقات المخفية في معظم الشبكات العصبية العميقة ,مزاياها في الكفاءة الحسابية وحل تلاشي التدرج يجعلها الخيار الأفضل رغم عيوبها .

مقارنة بين دوال التنشيط :

وجه المقارنة	step	sigmoid	RelU
السلوك	حاد ثنائي (0 أو 1)	ناعم احتمالي بين 0 و 1	بسيط , خطي للمدخلات الموجبة
النعومة	غير ناعمة	ناعمة جداً	ناعمة (ماعدا الصفر)
قابلية التفاضل	لا (ميلها صفر أو غير معرف)	نعم في كل مكان	نعم ميلها 1 لـ $x > 0$
مشكلة تلاشي التدرج	لا تنطبق	مشكلة كبيرة	تحلها بشكل كبير
التكلفة الحسابية	منخفضة جداً	مرتفعة (أسي)	منخفضة جداً
الاستخدام الشائع	البرسيبترون	طبقة الإخراج	الطبقات المخفية

النهاية